

NAME OF STUDENT: Muhammad Danish ID No: 470-2024

BASIC 8086 PROGRAMMING

Objectives:

- To understand data transfer and exchange using MOV and XCHG instructions.
- To work with DB, DW, DD variables and appropriate registers.
- To understand the difference between value and address using LEA.

Assembly language directly works with the processor's registers and memory.

A **register** is a very small and very fast storage location inside the CPU.

MOV INSTRUCTION:

The most important data-transfer instruction is:

MOV destination, source // Copy the source value into the destination.

Example:

MOV AX, 10 //AX=10

MOV BX, AX // BX = AX (The value in AX is copied to BX.)

MOV does not remove the original value.

MOV AX, 10

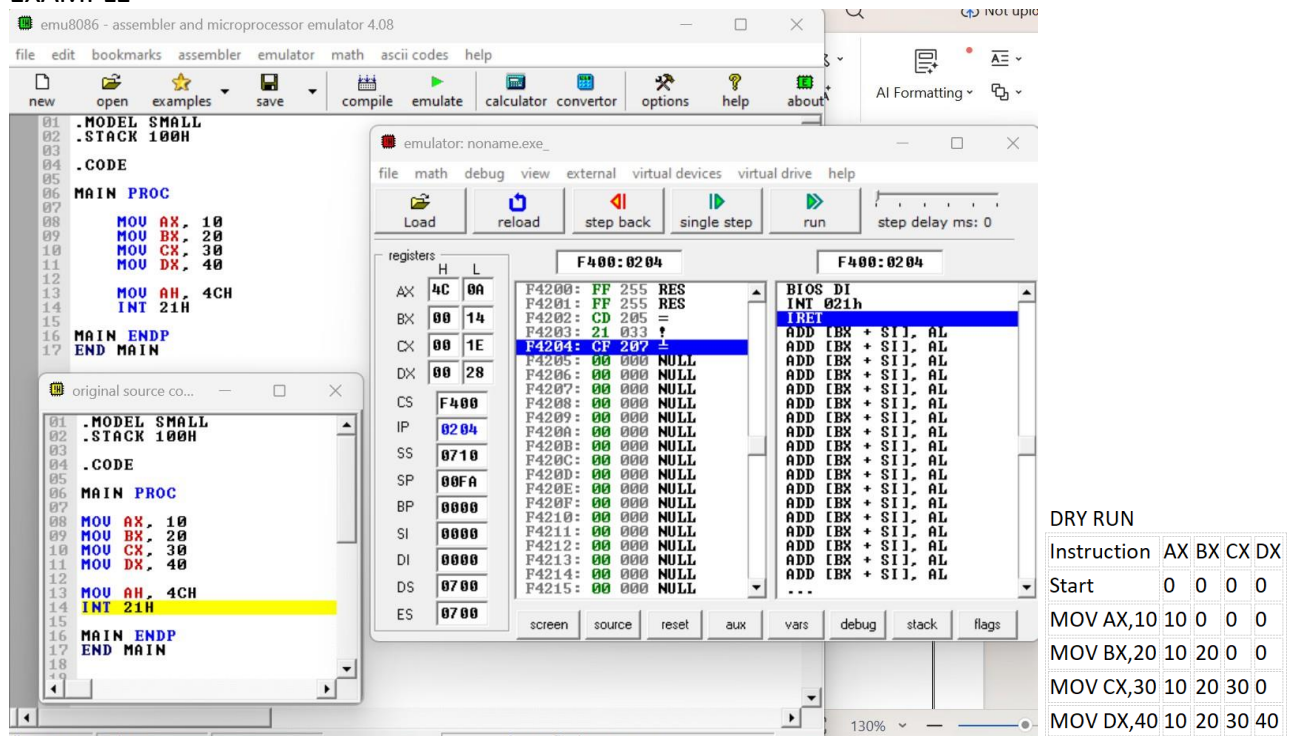
MOV BX, AX

After execution:

AX = 10

BX = 10

EXAMPLE



The screenshot shows the emu8086 interface with the following assembly code in the main window:

```

01 .MODEL SMALL
02 .STACK 100H
03
04 .CODE
05
06 MAIN PROC
07
08     MOV AX, 10
09     MOV BX, 20
10     MOV CX, 30
11     MOV DX, 40
12
13     MOV AH, 4CH
14     INT 21H
15
16 MAIN ENDP
17 END MAIN

```

The registers window shows the following values:

Register	H	L
AX	4C	0A
BX	00	14
CX	00	1E
DX	00	28
CS	F4	00
IP	02	04
SS	07	10
SP	00	FA
BP	00	00
SI	00	00
DI	00	00
DS	07	00
ES	07	00

The instruction list window shows the following instructions:

Address	Instruction
F42000	FF 255 RES
F42001	FF 255 RES
F42002	CD 205 =
F42003	21 033
F42004	CF 207
F42005	00 000 NULL
F42006	00 000 NULL
F42007	00 000 NULL
F42008	00 000 NULL
F42009	00 000 NULL
F4200A	00 000 NULL
F4200B	00 000 NULL
F4200C	00 000 NULL
F4200D	00 000 NULL
F4200E	00 000 NULL
F4200F	00 000 NULL
F42010	00 000 NULL
F42011	00 000 NULL
F42012	00 000 NULL
F42013	00 000 NULL
F42014	00 000 NULL
F42015	00 000 NULL

The DRY RUN window shows the following execution results:

Instruction	AX	BX	CX	DX
Start	0	0	0	0
MOV AX,10	10	0	0	0
MOV BX,20	10	20	0	0
MOV CX,30	10	20	30	0
MOV DX,40	10	20	30	40

Faculty of Computing and Information Technology (FCIT)
Department of Computing Indus University, Karachi

NAME OF STUDENT: Muhammad Danish **ID No:** 470-2024

MOV WITH DIFFERENT TYPES

Immediate to register

MOV AX, 25 // AX = 25

Register to register (8-bit)

MOV AL, 25

MOV BL, AL //BL=AL

After execution:

AL = 25

BL = 25

SIZE MUST MATCH

bit data → 8-bit register

16-bit data → 16-bit register

EXAMPLE

Instruction	AX	BX	CX	DX
Start	0	0	0	0
MOV AX,25	25	0	0	0
MOV BX,AX	25	25	0	0
MOV CX,BX	25	25	25	0
MOV DX,CX	25	25	25	25

XCHG INSTRUCTION

XCHG is used to exchange/swap two values.

XCHG destination, source

Example:

MOV AX, 10 //AX=10

MOV BX, 20 //BX=20

XCHG AX, BX

After execution:

AX = 20

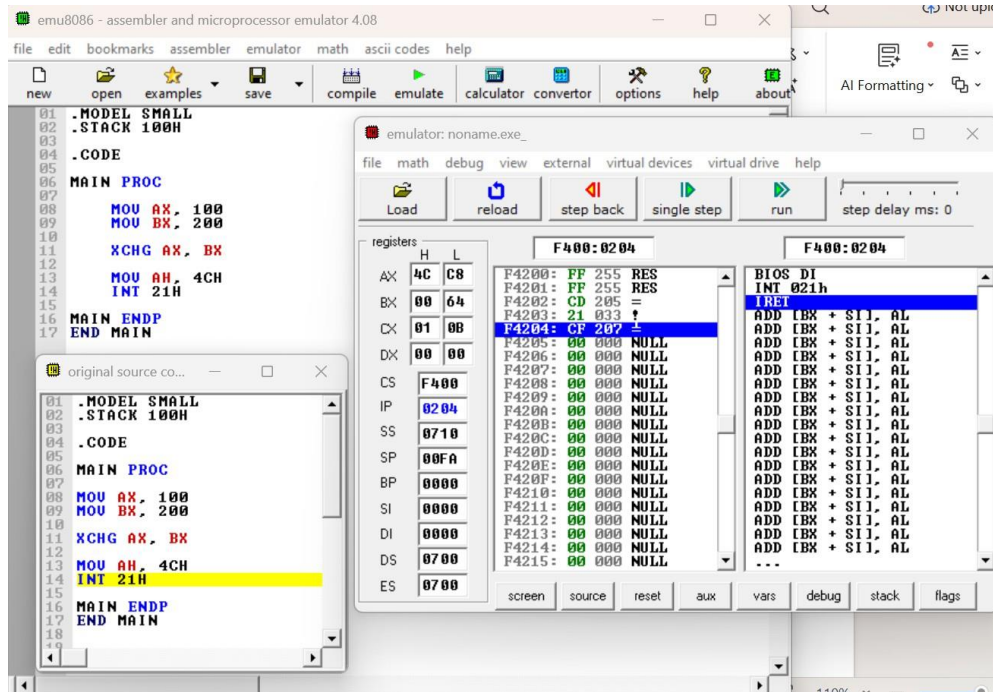
BX = 10

Faculty of Computing and Information Technology (FCIT)

Department of Computing Indus University, Karachi

NAME OF STUDENT: Muhammad Danish ID No: 470-2024

EXAMPLE



DRY RUN

Instruction	AX	BX	CX	DX
Start	0	0	0	0
MOV AX,100	100	0	0	0
MOV BX,200	100	200	0	0
XCHG CX,BX	200	100	0	0

DEFINE BYTE (DB)

First, introduce variables.

NUM1 DB 10 // Stores the value 10 in one byte.

Example:

.DATA

NUM1 DB 10

NUM2 DB 20

MOV AL, NUM1 //AL=10

MOV BL, NUM2 //BL=20

DEFINE WORD (DW)

Example:

NUM1 DW 1234H //A word is 16-bit.

MOV AX, NUM1 //Valid.

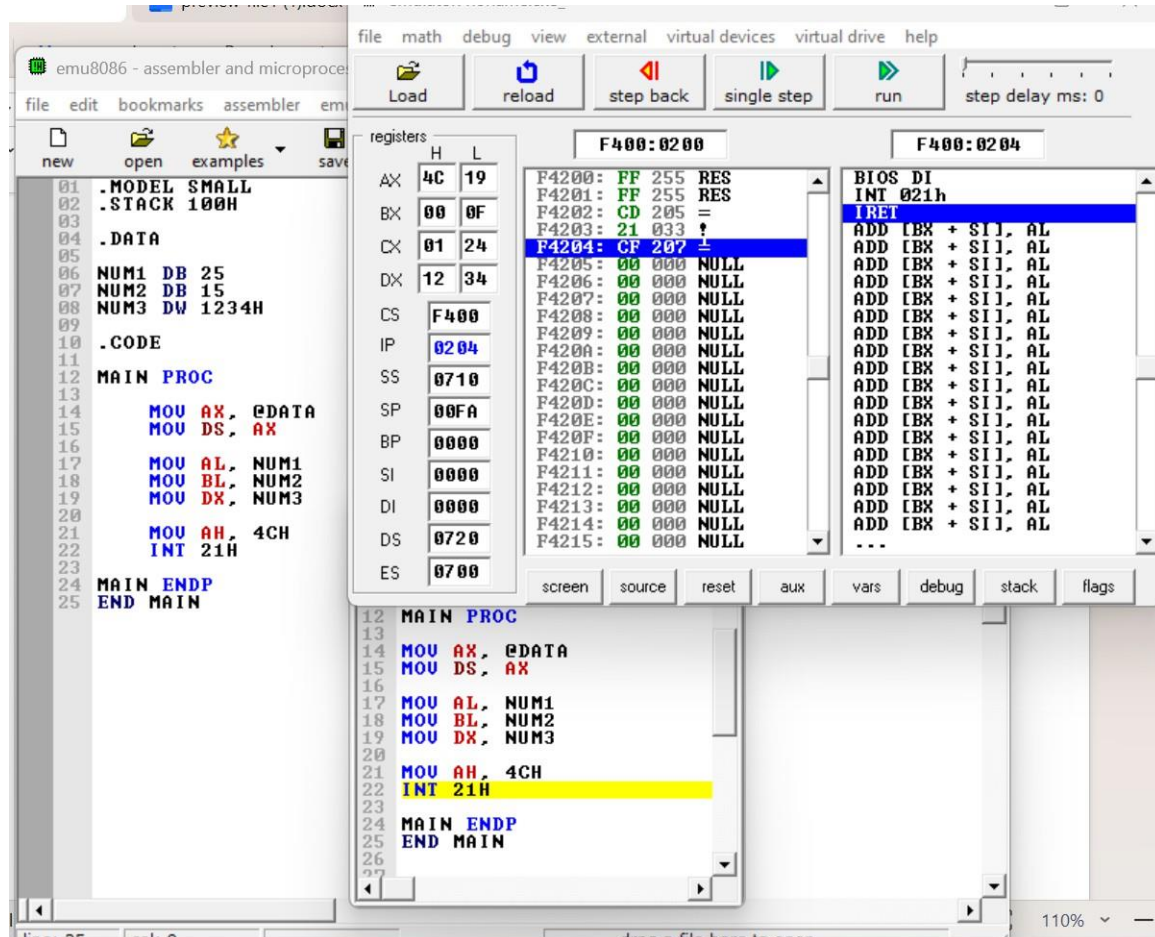
MOV AL, NUM1 // INVALID

Declaration	Size	EXAMPLE
DB (Define Byte)	8-bit / 1 byte	10
DW (Define Word)	16-bit / 2 bytes	1000
DD (Define Doubleword)	32-bit / 4 bytes	12345678H

Faculty of Computing and Information Technology (FCIT)
Department of Computing Indus University, Karachi

NAME OF STUDENT: Muhammad Danish **ID No:** 470-2024

EXAMPLE



VALUE VS ADDRESS

MSG DB 'HELLO\$' //MSG represents the memory location where the string "HELLO\$" is stored.
MOV DX, MSG

- Value/Data: 'HELLO\$' — the data stored in MSG.
- Address: The memory location where MSG is stored.

LEA DX, MSG // Loads the address of MSG into DX.

OR

MOV DX, OFFSET MSG // Loads the address of MSG into DX.

Load Effective Address (LEA)

Example:

LEA SI, MSG // Loads the address of MSG into SI.

.DATA

MSG DB 'HELLO\$'

LEA DX, MSG // The address of the string is loaded into DX.

TASK 1

Write a program that:

Stores any values in AX and BX.

Transfers the value of AX to CX.

Transfers the value of BX to DX.

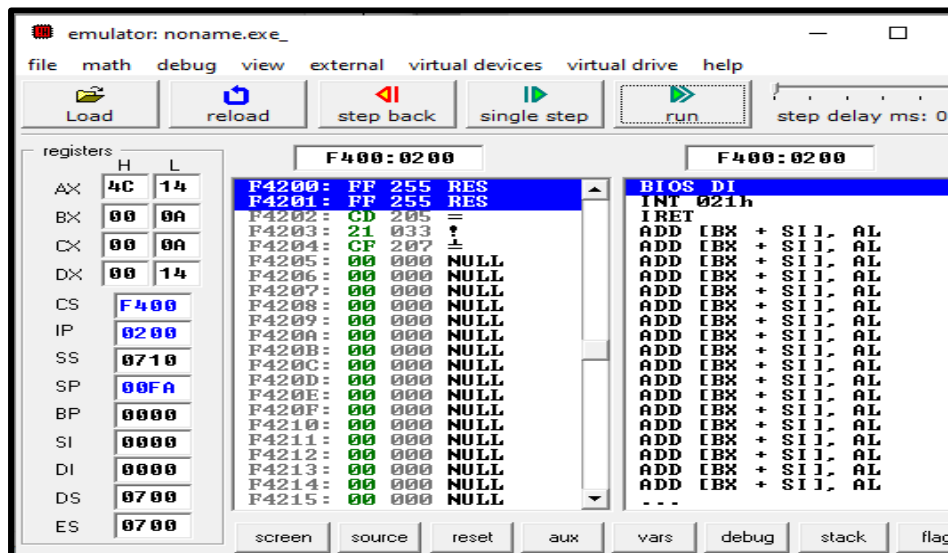
Swaps the values of AX and BX using XCHG.

Observe all four registers after execution.

CODE:

```
01  
02 .MODEL SMALL  
03 .STACK 100H  
04 .CODE  
05 MAIN PROC  
06     MOV AX, 10  
07     MOV BX, 20  
08     MOV CX, AX  
09     MOV DX, BX  
10     XCHG AX, BX  
11     MOV AH, 4CH  
12     INT 21H  
13 MAIN ENDP  
14 END MAIN  
15
```

OUTPUT:



TASK 2

Declare: Two variables using DB. Two variables using DW. Load the byte variables into appropriate 8-bit registers and the word variables into appropriate 16-bit registers. Observe the register values and explain why different register sizes are used.

OUTPUT:

```

assembler status
external view

there are errors.
(11) wrong parameters: MOV AL, NUM2
(11) operands do not match: 8 bit register and 16 bit address (address)
    
```

CONCLUSION: The assembler gives an error "operands do not match" and the program does not compile or run. No register changes, because the instruction is never executed.

TASK 4

Declare a string variable: MSG DB 'HELLO\$' Load the address of MSG into a register using LEA. Then try to load the value/data of a variable into a register using MOV. Observe both registers.

Question: What is the difference between the value and the address?

CODE:

```

01 .MODEL SMALL
02 .STACK 100H
03 .DATA
04 MSG DB 'HELLO$'
05 .CODE
06 MAIN PROC
07     MOV AX, @DATA
08     MOV DS, AX
09
10     LEA DX, MSG      ; DX = ADDRESS of MSG
11     MOV AL, MSG      ; AL = VALUE <first character> of MSG
12
13     MOV AH, 4CH
14     INT 21H
15
16 MAIN ENDP
17 END MAIN
    
```

OUTPUT:

registers		0721:000B		0721:000B	
	H L				
AX	07 48	07210: B8 184 7		MOV AX, 00720h	
BX	00 00	07211: 20 032 SPA		MOV DS, AX	
CX	01 1F	07212: 07 007 BEEP		MOV DX, 00000h	
DX	00 00	07213: 8E 142 A		MOV AL, [00000h]	
CS	0721	07214: D8 216 1		MOV AH, 04Ch	
IP	000B	07215: BA 186 11		INT 021h	
SS	0710	07216: 00 000 NULL		NOP	
SP	0100	07217: 00 000 NULL		NOP	
BP	0000	07218: A0 160 A		NOP	
SI	0000	07219: 00 000 NULL		NOP	
DI	0000	0721A: 00 000 NULL		NOP	
		0721B: B4 180 1		NOP	
		0721C: 4C 076 L		NOP	
		0721D: CD 205 =		NOP	
		0721E: 21 033 7		NOP	
		0721F: 90 144 E		NOP	
		07220: 90 144 E		NOP	
		07221: 90 144 E		NOP	
		07222: 90 144 E		NOP	
		07223: 90 144 E		NOP	

Answer: The value is the data stored in MSG ('HELLO\$', starting with 'H'). The address is the memory location where MSG is stored.

TASK 5

Write a program using MOV and XCHG with AX, BX, CX, and DX. Before running the program, make a dry-run table and predict the final values of all registers. Then run the program in EMU8086 and compare your answer with the actual result.

DRY-RUN(Predicted)

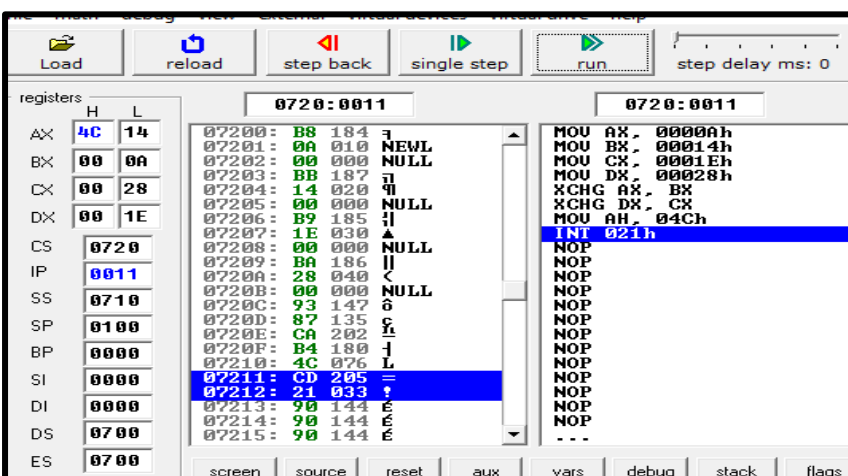
Instruction	AX	BX	CX	DX
Start	0	0	0	0
MOV AX,10	10	0	0	0
MOV BX,20	10	20	0	0
MOV CX,30	10	20	30	0
MOV DX,40	10	20	30	40
XCHG AX,BX	20	10	30	40
XCHG CX,DX	20	10	40	30

Code:

```

01
02 .MODEL SMALL
03 .STACK 100H
04 .CODE
05 MAIN PROC
06     MOV AX, 10
07     MOV BX, 20
08     MOV CX, 30
09     MOV DX, 40
10     XCHG AX, BX
11     XCHG CX, DX
12     MOV AH, 4Ch
13     INT 21H
14 MAIN ENDP
15 END MAIN
    
```

OUTPUT:



Comparison: The predicted final values (AX=20, BX=10, CX=40, DX=30) match the actual values shown in EMU8086 (AX=0014H, BX=000A H, CX=0028H, DX=001EH in hex).

HELLO